

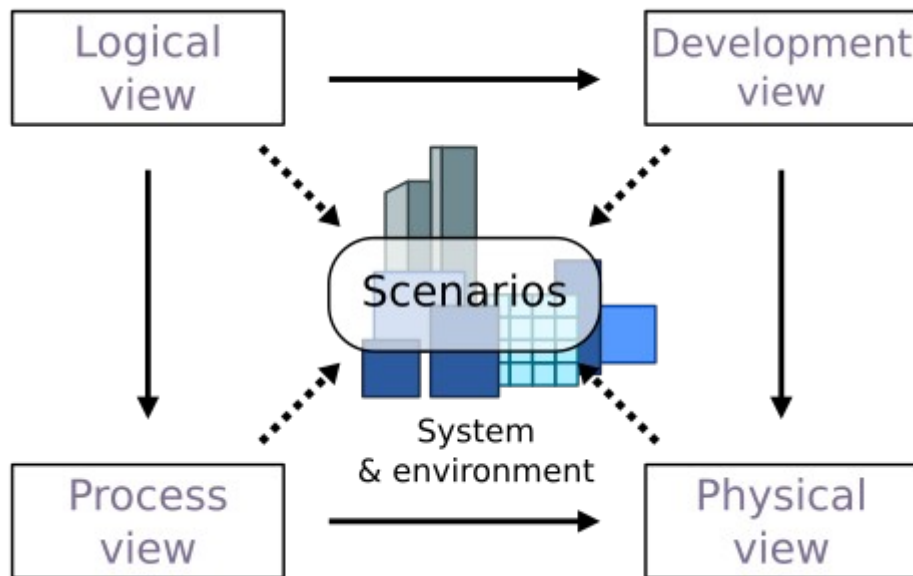
Table of Contents

1 Architectural view model.....	3
1.1 The logical view.....	3
1.2 The Physical view.....	3
1.3 The Development view.....	4
1.4 The Process view.....	4
1.5 The Scenarios (or use case view).....	4
2 Architectural guidelines.....	5
2.1 Loosely coupled architecture.....	5
2.2 Asynchronous component communication.....	5
3 The Deployment view.....	6
3.1 Software artifacts.....	6
3.1.1 Folder structure.....	6
3.1.2 Github.....	6
3.1.3 Music sources.....	7
3.1.4 Self-documenting python code.....	7
3.2 Target environment.....	8
3.3 Software installation.....	9
3.3.1 Development: install script.....	9
3.3.2 Production.....	9
3.4 Runtime.....	9
3.5 Configuration.....	9
3.6 System services.....	9
3.7 Plugins.....	10
3.7.1 Project structure.....	11
3.7.2 Plugin base class.....	11
3.7.3 Dynamic Plugin loading.....	11
3.7.4 life-cycle handler.....	12
4 The Logical view: Class diagrams.....	13
4.1 Introduction on Class diagrams.....	13
4.1.1 UML class notation.....	13
4.1.2 UML Class relationships.....	14
4.2 The top level diagram.....	16
4.3 Messaging.....	17
4.4 Remote Monitoring Service.....	19
4.5 USB Service.....	20
4.6 Wifi Service.....	21
4.7 Logging.....	23
4.7.1 Logging introduction.....	23
4.7.2 Logging design choices.....	23
4.7.3 Loggers.....	24
4.7.4 Queue handling.....	24
4.7.5 Fallback loggers.....	25
4.7.6 LogHealthMonitor.....	25
4.7.7 Filter.....	25
4.7.8 Formatter.....	25
4.7.9 Linux Logrotate service.....	26
4.7.10 LogHealthMonitor.....	26
4.7.11 Class diagram.....	27
4.7.11.1 falthandler.....	28
4.7.11.2 SafeLogger class.....	28

4.7.11.3 StrictLogHandler.....	28
4.7.11.4 logging.Handler.....	28
4.7.11.5 logging.Logger.....	28
4.7.11.6 ColorFormatter.....	29
4.7.11.7 uvicorn integration.....	29
5 The Logical view: State diagrams.....	30
5.1 Maincontrol state.....	30
5.2 Oradio Control States.....	31
5.3 Access Point state control.....	32
5.4 PluginManager State control.....	33
6 The Physical view: Component diagrams.....	35
6.1 OS Service.....	35
6.2 Messaging (Commands).....	36
6.3 Music Players.....	37
6.4 GPIO components.....	38
7 The Process view: Sequence diagrams.....	39
7.1 Messaging.....	39
8 The scenario view: use case diagrams.....	40

1 Architectural view model

- 5 The architecture 4+1 views are used to describing the software systems based on different viewpoints, to discover if the architecture fulfills the needs of the different stakeholders. The views will be presented using the Unified Modeling Language (UML) tool



llibre

1.1 The logical view

- 10 The Logical View, often referred to as the “Class Diagram”. It focuses on the static structure of the system. This view deals with the essential components of the system, such as classes, objects, relationships, and their attributes. Class diagrams visually represent the classes and their associations in the system, to understand how various components interact with one another. This view provides insights into the overall design and organization of the system’s functionality.
- 15 Related UML diagrams:
- Class diagrams
 - Component diagram

1.2 The Physical view

- It concerns the topology of the software components on the physical infrastructure as well as the physical connections between components. It includes details about servers, nodes, and their interconnections, including defining the communication protocols This view ensures that the system architecture aligns with the physical constraints and requirements.
- Related UML diagrams:
- Class diagrams
 - State diagrams
 - Component diagrams
- 25

1.3 The Development view

30 The system from the perspective of the programmers (or the implementation view) and is concerned with software management. It maps the logical design to physical code organization, clarifies module boundaries, dependencies, and re-usability.

1.4 The Process view

The dynamic aspects of the system, explains the system processes and how they communicate. Models the dynamic behavior, execution flow, threading, synchronization, performance, and fault tolerance.

35 Related UML diagrams:

- Sequence diagrams,
- Activity diagrams
- State diagrams

1.5 The Scenarios (or use case view)

40 It captures what the system does from an end-user perspective. The description of an architecture is illustrated using a small set of use case, or scenarios, which become a fifth view. The scenarios describe sequences of interactions between objects and between processes. They are used to identify architectural elements and to illustrate and validate the architecture design.

Related UML diagrams:

- 45
- Use Case diagram

2 Architectural guidelines

2.1 Loosely coupled architecture

50 The architecture should follow the guidelines for loosely coupled architecture. A loosely coupled architecture is a design principle in which system components are weakly associated. This means that each component can operate independently, allowing for greater flexibility in system design and maintenance.

Key characteristics:

- 55
- Changes made to one component have minimal impact on others, which enhances the overall resilience of the system
 - Each component has little or no knowledge of the internal workings of other components.
 - Systems can scale more easily as components can be modified or replaced independently.
 - Each component can be developed, deployed, tested and maintained independently

60 2.2 Asynchronous component communication

A loosely coupled architecture using a queue allows different components of a system to communicate asynchronously, meaning they can operate independently without being directly connected.

65 The component communication uses the publish/subscriber model, in which a communication queue will be assigned to each of the subscribers. When a new message is published, the related message Topic will be distributed to the assigned subscribers queue.

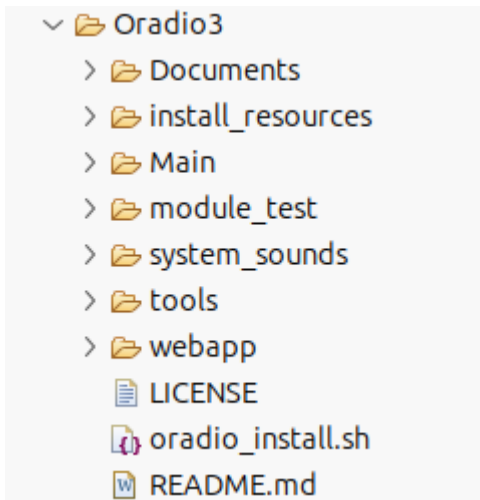
Key characteristics:

- 70
- Independence: Producers and consumers don't need to know about each other. They only interact with the queue, reducing direct dependencies.
 - Failure isolation: If one component fails, the queue allows other components to continue functioning. This isolation minimizes the impact of failures on the overall system.
 - Asynchronous Communication: Components can communicate without waiting for each other to respond, which reduces downtime and improves responsiveness.
 - Buffering: They act as buffers that store messages until the receiving service is ready to process them. This helps manage varying loads and prevents service overload
- 75

3 The Deployment view

3.1 Software artifacts

3.1.1 Folder structure



The folder structure of the Oradio project :

- Documents: for design documentation
- install_resources: resources related to configuration files, system service file, shell scripts required for installation of Oradio project.
- Main: the python modules
- module_test: module test python programs
- system_sounds: sound files (wav) for system related sound (e.g. click.wav for touch button click sound)
- tools: shell scripts
- webapp: resources required for the web-interface.
- LICENSE: The GNU general public license, which is required to include to inform users of their rights and obligations regarding the software. This ensures proper attribution and compliance with the terms of the license.
- oradio_install.sh: install script to install a fresh Oradio project from scratch
- README.md: install instructions

3.1.2 Github

GitHub is a cloud-based platform for hosting and managing code that allows developers to collaborate on projects using version control. It is primarily used for storing code, tracking changes, and facilitating teamwork on software development projects. In this project github will be used for:

- Version Control: to track changes in code over time.
- Repositories: to store code, enabling sharing of code and collaboration
- Pull Requests: method to propose changes to a project, allowing for code review and discussion before integration.
- Issue tracking: to track bugs and feature requests, to manage the projects workflow.

3.1.3 Music sources

The Oradio is a music player that can play music from different sources:

- Music that is provided on the USB memory device.
- A music stream from a web-radio
- A music stream from Spotify player application

3.1.4 Self-documenting python code.

Module headers should have following Google style docstring formatting:

```
"""
    #####          ##          #####          #          #####
    #   #   #   #   #   #   #   #   #   #   #   #   #
    #   #   #   #   #   #   #   #   #   #   #   #
    #   #   #####   #####   #   #   #   #   #   #
    #   #   #   #   #   #   #   #   #   #   #   #
    #####   #   #   #   #   #####   #   #####

Created on May 28, 2026
@author:      Henk Stevens & Olaf Mastenbroek & Onno Janssen
@copyright:   Copyright, Oradio Stichting
@license:     GNU General Public License (GPL)
@organization: Oradio Stichting
@version:     1
@email:       oradioinfo@stichtingoradio.nl
@status:      Development
@summary:     One line summary of the module
@description: Detailed description of module
"""
```

- @author: the author(s)
- @copyright, @license, @organization, @version, @email, @status: open-source related information.
- @summary: one-line summary of module
- @description: detailed description of modules
- For each class, method or function there should be Google style documentation strings (docstring) , with following format:

```
def example_function(param1, param2):
    """Summary line.

    Detailed description of the function's purpose and behavior.

    Args:
        param1 (int): Description of the first parameter.
        param2 (str): Description of the second parameter.

    Returns:
        bool: True if successful, False otherwise.

    Raises:
        ValueError: If param1 is negative.

    Examples:
        >>> example_function(1, 'test')
        True
    """
```

- **Summary line:** A brief description of the module, class, or function, limited to one line.
- **Detailed Description:** This section provides a more comprehensive explanation of the code's purpose and usage.
- **Sections:**

Section	Description
Args	Lists the parameters accepted by the function, including their types.
Returns	Describes the return value and its type.
Raises	Lists exceptions that the function may raise, along with conditions.
Examples	Provides usage examples to illustrate how to use the function or class.

3.2 Target environment

- The RaspberryPi-3A+ headless module will be used as central control unit, providing hardware for:
 - Connectivity: WiFi, Bluetooth, Ethernet, USB
 - Input/Output pinning (GPIO): for user interface, as buttons and led indications
 - RaspiOS: Raspberry Pi OS (open-source) is a Unix-like operating system developed for the Raspberry Pi line of single-board computers. Based on Debian, a Linux distribution, it is maintained by Raspberry Pi Holdings and optimized for the Pi's hardware, with low memory requirements and support for both 32-bit and 64-bit architectures.
 - Extension interface at GPIO for Audio Playback (DigiAmp), touch buttons, volume knob and led drivers.
- Raspberry Pi OS: latest version, after being tested by developers

- Python is the programming environment, which has a wide range of libraries specifically designed for hardware interaction, created by other developers, eliminating to write everything from scratch. Comes preinstalled on the Raspi-OS

145 **3.3 Software installation**

3.3.1 Development: install script

The install shell script will install and configure the environment for the Oradio:

- logging: create logging folders
- services: install services, using the service templates in install_resources folder. Enable and start these services.
- Linux packages: install required Linux packages
- Python packages: install required Python packages
- Configuration: adapt configuration files

150

3.3.2 Production

For production an image is created from a released software version. The “scrum-master” is responsible for handing over an image for production.

155

3.4 Runtime

- runs on boot
- logging
- handling errors

160

3.5 Configuration

- Configuration items:
 - config.txt,
 - asound.conf,
 - mpd.conf,
 - logrotate.conf
- ??

165

3.6 System services

- systemd services:
 - librespot.service

170

- mpd.service
- oradio.service
- usb_low_idle_power_service: (still needed??)
- Dynamic device management: Udev rules for USB devices

175

3.7 Plugins

The Oradio should be prepared for adding new functionalities without or with minimal consequences for the basic Oradio. The basic Oradio offers following services:

- Touch button handling
- 180 • Led indication handling
- System sound for announcements
- Internet connectivity (WiFi)
- Web-interface to configure or customize the Oradio
- USB support to import music files and playlists
- 185 • Music player: Music Player Daemon (MPD)

The basic state-machine is setup to handle above listed services. The objective is to maintain the python code as a one-code-base. Additional services should be offered as a plugin, enabling dynamically loading and executing the new functionality, without affecting the basic state-machine. Example for plugins:

- 190 • Spotify streaming services
- Bluetooth speaker services
- Presence sensor services
- Bluetooth based light control.

The plugins module should be available in the code base. Via the web-interface a plugin can be enabled/disabled.

195

Python based plugins require a plugin framework with following elements:

- a project structure for the plugins
- a plugin class (or interface)
- a loader

```
/Main
==> /plugins/
    plugin_module_1.py
    plugin_module_2.py
    plugin_module_n.py
==> /core/
    plugin_loader.py
oradio_control.py
touch_buttons.py
etc
```

```

/Main
==> /plugins/
    plugin_module_1.py
    plugin_module_2.py
    plugin_module_n.py
==> /core/
    plugin_loader.py
oradio_control.py
touch_buttons.py
etc

```

The plugins directory holds the plugins modules. The plugin_loader is in the /core folder, which provides a base class (or interface) that all plugins must implement, to ensure consistency.

3.7.2 Plugin base class

205 The base class should provide following methods:

```
class PluginBase:
```

```
    def initialize(self,config:dict)
```

Called once when plugin is loaded, to setup the plugin

```
    @abstractmethod
```

210 def run(self)

Main logic of the plugin, called by the framework to execute the plugin.

```
    def cleanup(self)
```

Called when the plugin is unloaded or shuts down, to cleanup resources

```
    def on_event(self, event:str)
```

215 Called upon an event to be processed by main state machine

```
    def health(self) → HealthStatus
```

returns the health status of plugin

```
    def on_error(exc:BaseException)
```

if exceptions arise around the plugin execution, route them to the method, for further processing

220

```

import importlib
import os
from pathlib import Path

def load_plugins(plugin_dir="plugins"):
    plugins = []
    for file in Path(plugin_dir).glob("*.py"):
        module_name = file.stem
        module = importlib.import_module(f"{plugin_dir}.{module_name}")
        if hasattr(module, "plugin") and isinstance(module.plugin, Plugin):
            plugins.append(module.plugin)
    return plugins

```

Each plugin should inherit from this class and implement the methods

3.7.3 Dynamic Plugin loading

Use Python's <importlib> to dynamically load plugins from plugins directory

```
import importlib
import os
from pathlib import Path

def load_plugins(plugin_dir="plugins"):
    plugins = []
    for file in Path(plugin_dir).glob("*.py"):
        module_name = file.stem
        module = importlib.import_module(f"{plugin_dir}.{module_name}")
        if hasattr(module, "plugin") and isinstance(module.plugin, Plugin):
            plugins.append(module.plugin)
    return plugins
```

225 3.7.4 life-cycle handler

The life-cycle handler could be organized in a state-machine, managing the plugins for loading, initialization, running and cleanup, from one place. In case a plugin crashes, the handler can catch the error and continue running the other plugins.

230 Important is that a plugin runs in its own environment, e.g. as a Thread. This allows plugins to run in parallel and a Thread can be stopped if needed with a stop signal

See also the [PluginManager State control section](#)

3.8 Non-Functional Requirements

Requirements Type	Description
Performance	Defines the speed and efficiency of the system, such as response time.
Reliability	Measures the system's ability to perform consistently over time.
Availability	Indicates the proportion of time the system is operational and accessible.
Maintainability	Refers to how easily the system can be updated or repaired.
Security	Encompasses measures to protect data and ensure authorized access.

Usability	Assesses how easy and intuitive the system is for users.
Scalability	Describes the system's ability to handle growth in users or data.
Compatibility	Evaluates how well the system integrates with other systems or components.
Extensibility	The ease with which new features can be added.
Recoverability	The ability to restore the system after a failure.

4 The Logical view: Class diagrams

The Oradio software layering typically consists of three main layers:

- the **hardware abstraction layer** (HAL), which includes the physical components.
- the **middle-ware layer**, which facilitates communication and data management between the hardware and applications. The OS services is considered to be part of this layer
- the **application layer**, where the actual software applications run and perform specific tasks.
- the **globals&utils layer**, which provide global and public functions

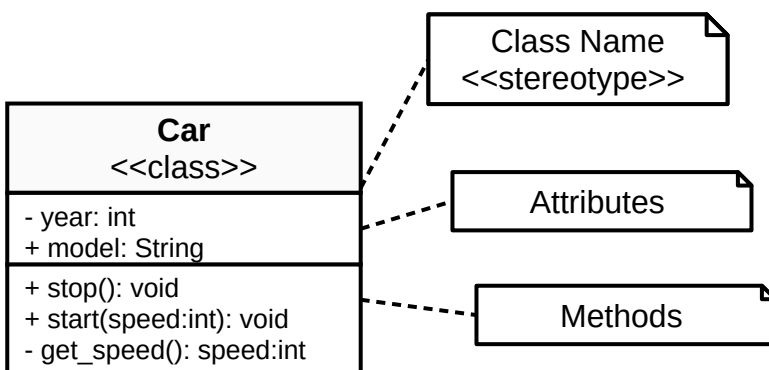
The diagrams are organized from a top-level view and for each python module the class design.

4.1 Introduction on Class diagrams

UML class diagrams are visual representations of the structure of a system, showing classes, their attributes, operations, and the relationships among objects. Class diagrams helps in modeling object-oriented systems and understanding how different components interact within the software.

4.1.1 UML class notation

Classes are depicted as boxes, each containing three compartments for the class name, attributes, and methods.



- **Class Name:** The name of the class is written in the top compartment of the class box and is centered and bold. The name may contain the name of the module in which the class is defined. E.g. <<class>>log_service.SafeLogger the class SafeLogger is defined in the module log_service. The <<stereotypes>> are used to extend the semantic of UML elements, providing additional information about the class. Examples of stereotypes:

- <<singleton>> : single instance class
- <<dataclass>>: data holder class
- <<static>>: a public class
- <<module>>: name of the (Python) module

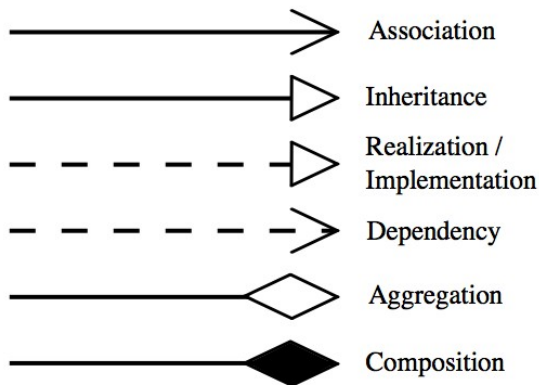
- **Attributes:** Attributes, also known as properties or fields, represent the data members of the class. They are listed in the second compartment of the class box and often include the visibility (e.g., (+) public, (-) private) and the data type of each attribute.

- **Methods:** Methods, also known as functions or operations, represent the behavior or functionality of the class. They are listed in the third compartment of the class box and

include the visibility (e.g., (+) public, (-) private), return type, and parameters of each method.

4.1.2 UML Class relationships

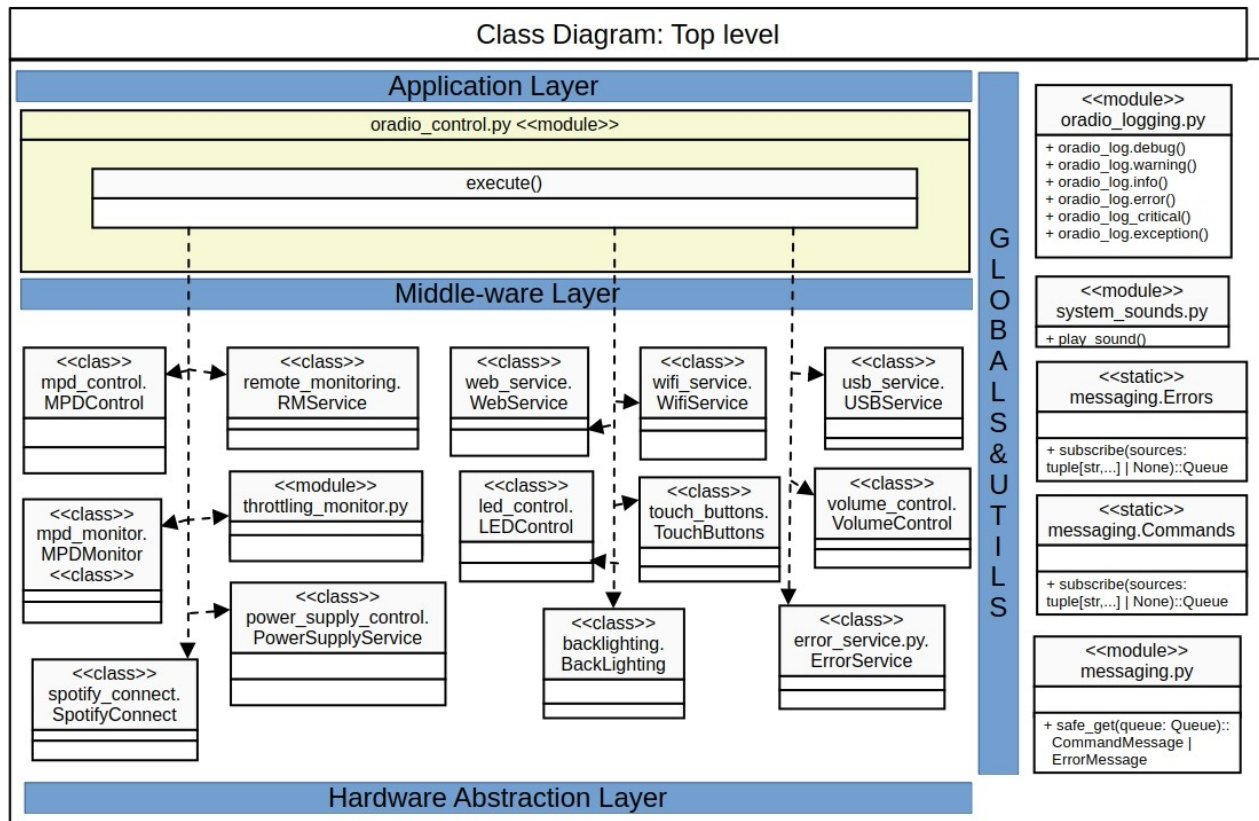
270 In class diagrams, relationships between classes describe how classes are connected or interact with each other within a system. Here are some common types of relationships in class diagrams:



- 275 • **Association:** An association represents a bi-directional relationship between two classes. It indicates that instances of one class are connected to instances of another class. Associations are typically depicted as a solid line connecting the classes, with optional arrows indicating the direction of the relationship.
- 280 • **Inheritance:** Inheritance represents an "is-a" relationship between classes, where one class (the subclass or child) inherits the properties and behaviors of another class (the superclass or parent). The subclass inherits all the functions of the parent class, and the parent class has all the attributes, methods, and subclasses.
- **Realization/Implementation:** Realization indicates that a class implements the features of an interface. It is often used in cases where a class realizes the operations defined by an interface.
- 285 • **Dependency:** A usage dependency ("uses") relationship in a UML class diagram indicates that one class (the client) utilizes or depends on another class (the supplier) to perform certain tasks or access certain functionality. The client class relies on the services provided by the supplier class but does not own or create instances of it.
- 290 • **Aggregation:** Aggregation is a specialized form of association that represents a "whole-part" relationship ("has"). It denotes a stronger relationship where one class (the whole) contains or is composed of another class (the part). In this kind of relationship, the child class can exist independently of its parent class.
- **Composition:** Composition is a stronger form of aggregation, indicating a more significant ownership or dependency relationship ("consists of"). In composition, the part class cannot exist independently of the whole class.

4.2 The top level diagram

There is one application running, being the python module `oradio_control.py`, which will initialize and create all instances necessary for the operation. The `execute()` function indicates that `oradio_control.py` is executed and running.



Most of the classes (stereotype `<<class>>`) will be instantiated during initialization phase of execution.

305 The `throttled_monitor` is globally instantiated, a monitoring thread will be started, to check the condition of the raspberry.

The global methods or functions are represented in the GLOBAL&UTILS layer, as these functions are public, being:

- `oradio_log.xx()`
- `play_sound()`
- `subscribe_commands()`, `subscribe_errors()`, `publish_command()`, `publish_error()`

310

4.3 Messaging

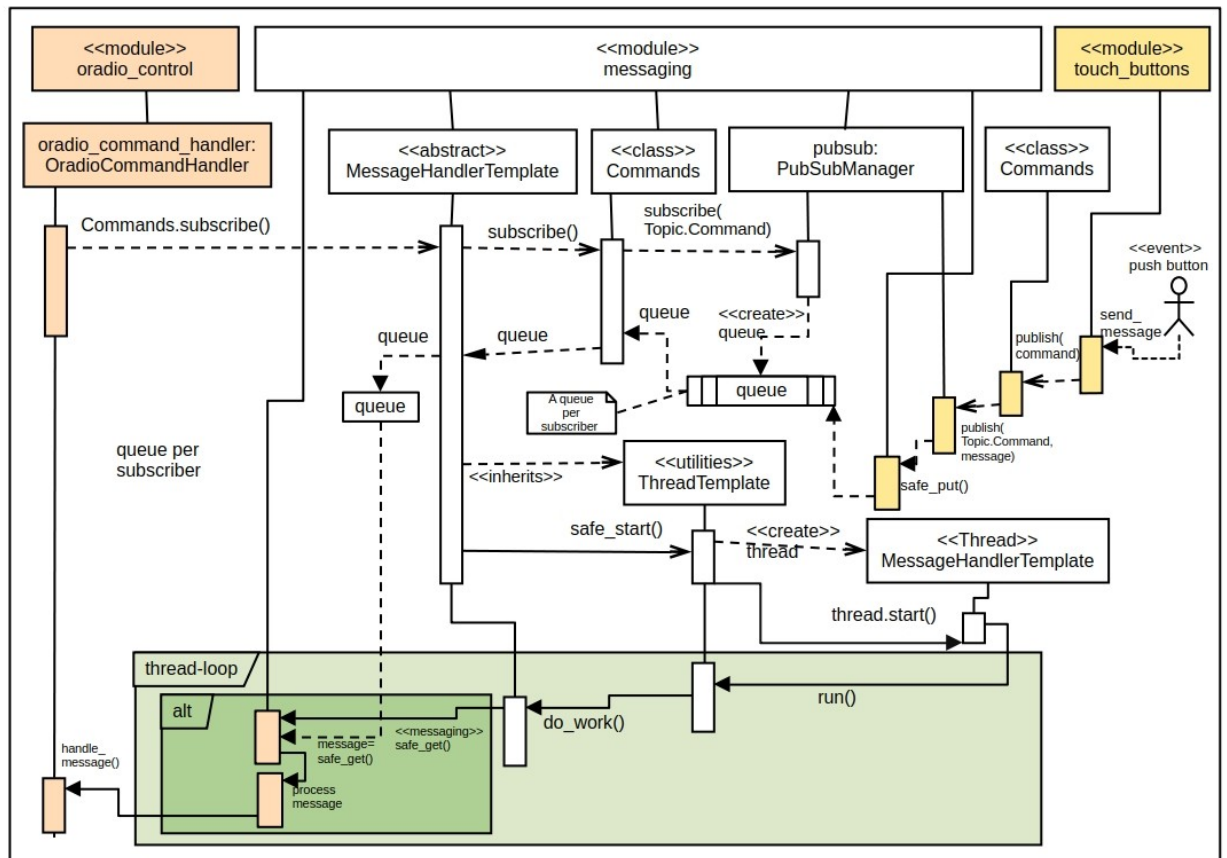
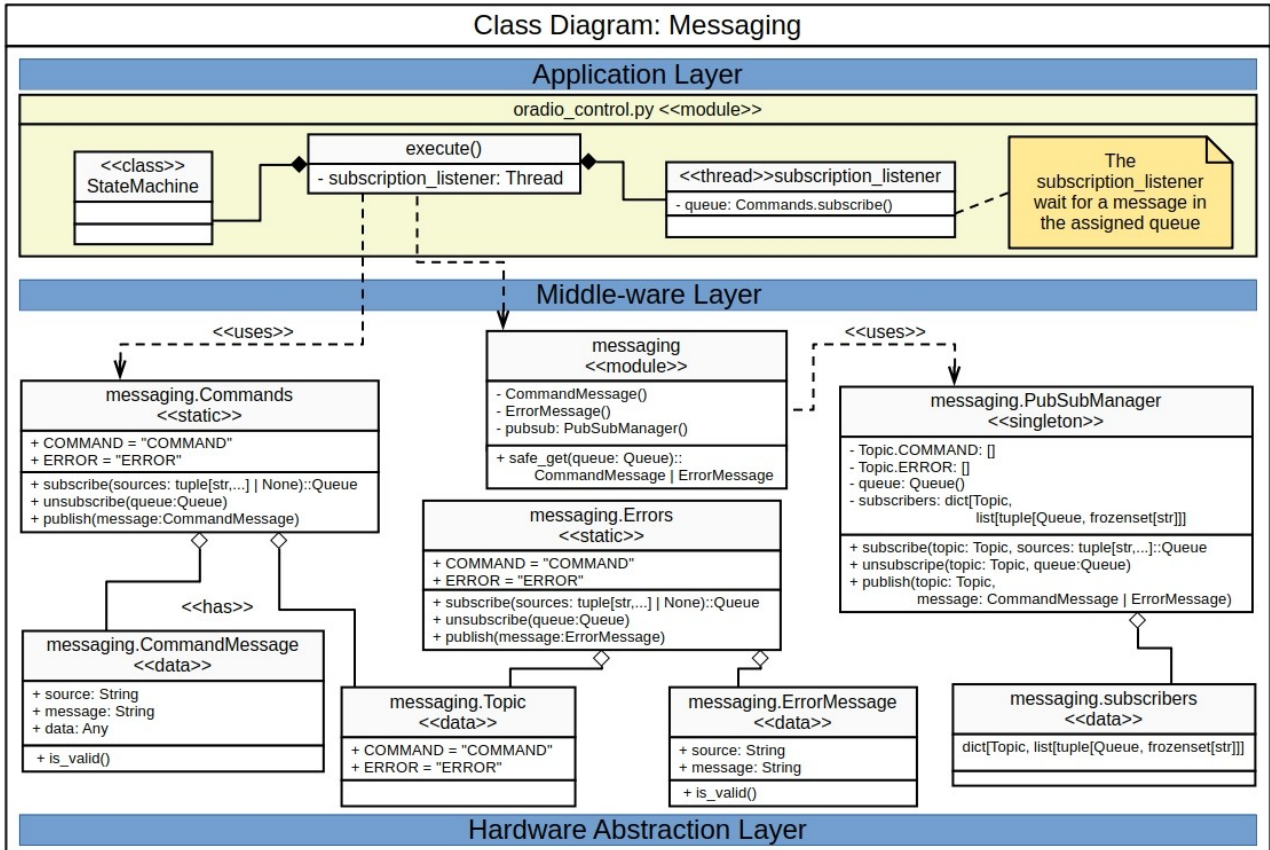


Figure 1: Messaging for Commands Topic

320 The messaging.py module provides services for the publish-subscribe model as inter-module communication. It is a messaging pattern where publishers send messages to a topic without knowing who the subscribers are, while subscribers express interest in specific topics to receive those messages. This model allows for asynchronous communication and decouples the components, enhancing scalability and flexibility in distributed systems.



The module has a public API to subscribe to a certain Topic or publish a message to a Topic. The defined Topics are:

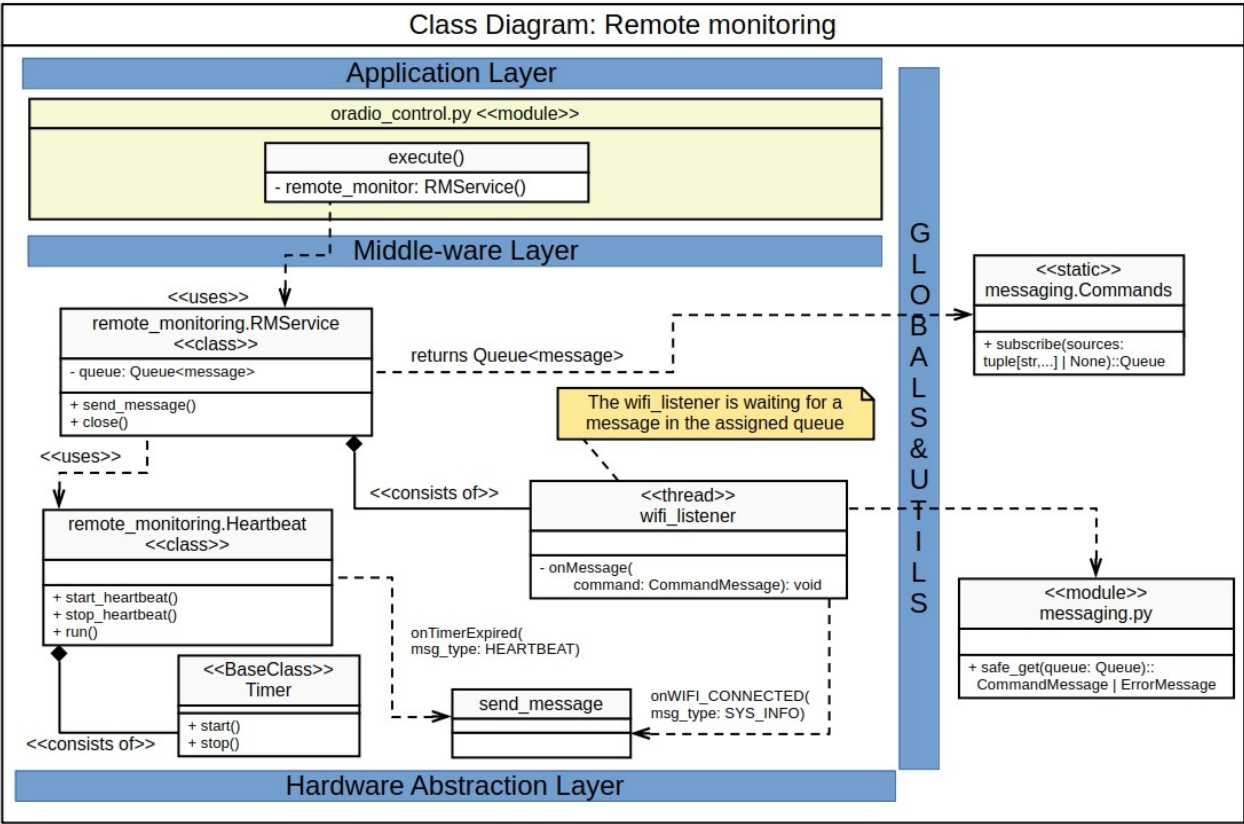
- **COMMAND:** representing a command
- **ERROR:** representing an error code

325 The public API provides following methods:

- `subscribe_commands()`: subscribe to Message Topic, returning a message queue to monitor for new commands
- `publish_command(message: CommandMessage)`: publish a command to COMMAND Topic
- `subscribe_errors()`: subscribe to Error Topic, returning a message queue to monitor for new errors
- `publish_error(message: ErrorMessage)`: publish a command to ERROR Topic

330

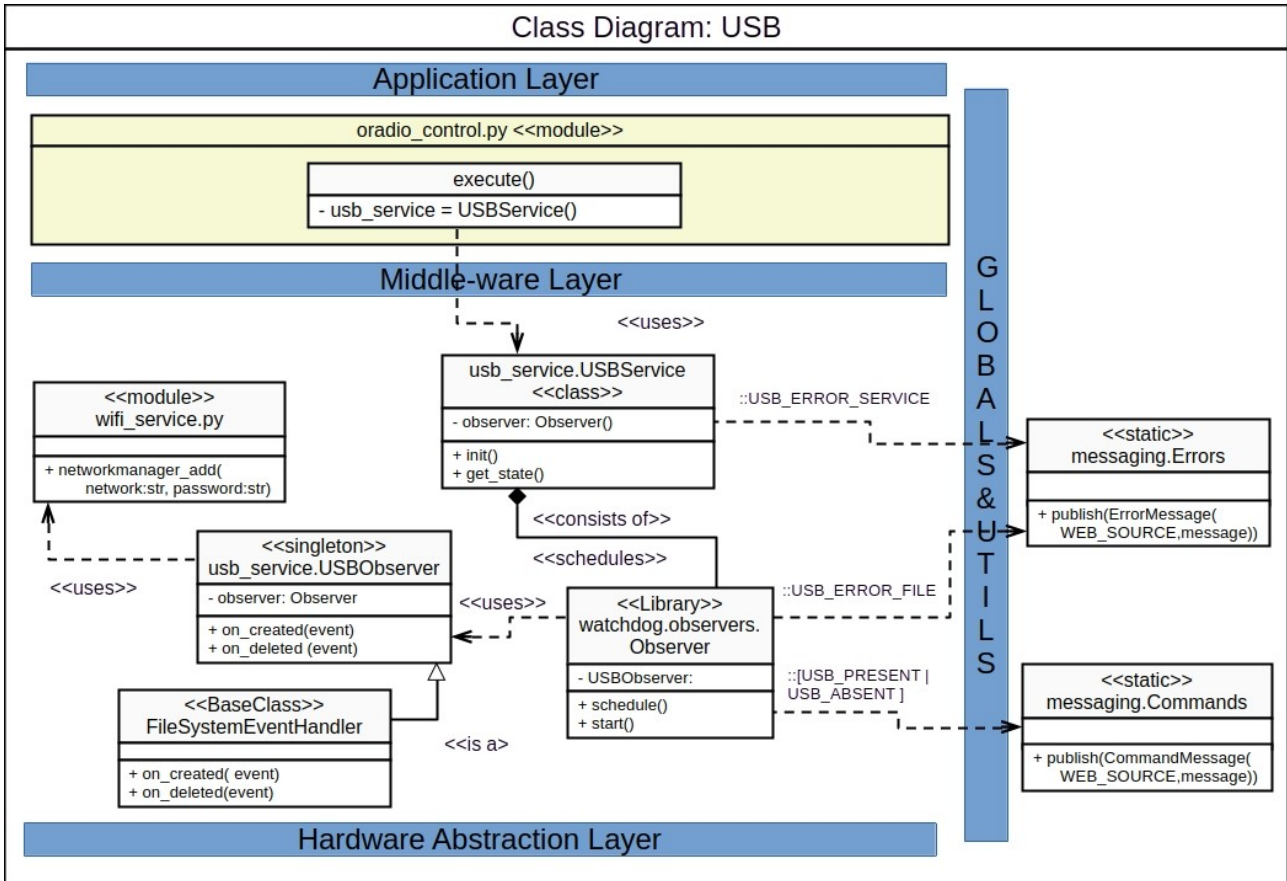
4.4 Remote Monitoring Service



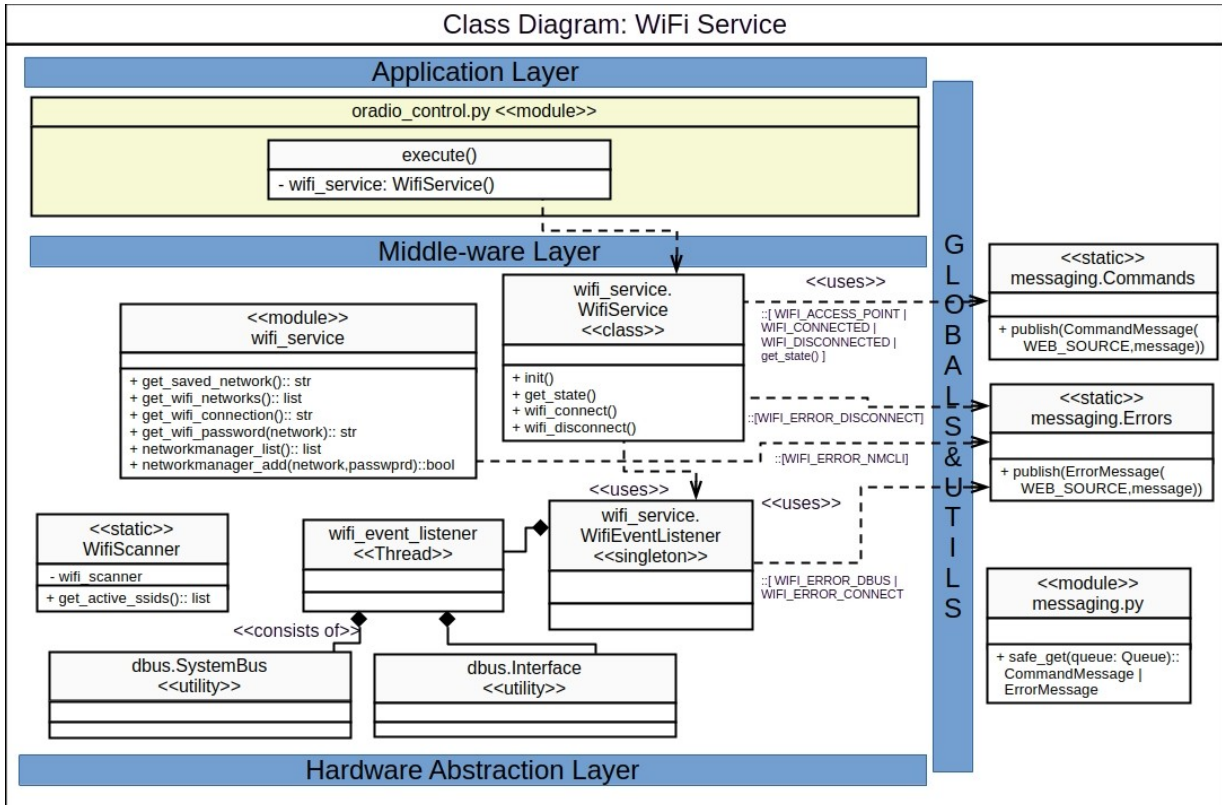
340

The Remote Monitoring Service maintains a connection with a Remote Server via Wifi. The class `RMService()` will be instantiated during initialization of `oradio_control.py`. The `RMService` class is depending on whether there is a WiFi connection available. So it initiates a thread `<wifi_listener>` that will listen to WiFi related commands, submitting a `subscribe_commands()`, which returns a message Queue. When WiFi is available is will send a `SYS_INFO` message to the server. A regular `HEARTBEAT` message is send to server as a keep-alive indication for the server

4.5 USB Service



4.6 Wifi Service



4.7 Logging

350

4.7.1 Non-Functional Requirements (NFR's)

Logging is categorized under reliability and maintainability. The key NFR's are:

NFR-ident	Requirement	Reference
NFR-LOG-001	The system shall log all items with a timestamp, severity level, service/component name, error code, and descriptive message.	Formatter
NFR-LOG-002	The application shall use Python's standard logging framework.	Python-logger
NFR-LOG-003	Logs shall use standardized levels such as DEBUG , INFO , WARNING , ERROR , and CRITICAL .	Filters
NFR-LOG-004	The logging system shall not reduce application performance by more than 5% (tbd) under normal load.	
NFR-LOG-005	The application shall not terminate because a logging handler fails.	Queue
NFR-LOG-006	Sensitive information, including passwords, authentication tokens, and personal data, shall not be written to logs.	
NFR-LOG-007	Logs shall be protected against unauthorized access and modification	
NFR-LOG-008	The system shall process at least 10 log events per second (tbd) without losing messages.	
NFR-LOG-009	Log files shall use rotation based on size or time.	Rotate
NFR-LOG-010	Rotated log files shall be removed according to the retention policy (tbd).	Rotate
NFR-LOG-011	All logs shall use asynchronous processing, using a QueueHandler and a QueueListener , where synchronous file or network handlers could block application requests.	Queue
NFR-LOG-012	The listener shall dispatch records to exactly one console output and exactly one file output.	Queue
NFR-LOG-015	The queue capacity shall be sufficient for the expected peak logging rate	Queue

NFR-LOG-016	Debug-level logging shall be disabled in production unless temporarily enabled for diagnosis.	
NFR-LOG-017	Logging shall continue functioning if the Python log service is temporarily unavailable, using logging mechanism provided by RaspiOS.	Fallback
NFR-LOG-018	Each application module shall use the Oradio-specific logger.	Loggers
NFR-LOG-019	Web-application (Uvicorn) logs shall be routed through the same QueueHandler as for the Application logs	Loggers
NFR-LOG-020	The system shall define behavior when the configured Python Queue is full (such as blocking, dropping log-records). A fallback method should be used to prevent losing the log-record	Fallback
NFR-LOG-021	The system shall publish an incident report in case when a log-record is not delivered to the configured Python Queue	Monitoring
NFR-LOG-021	The system shall write a Python traceback upon fatal signals in Python execution, for diagnostic purposes	Faulthandler

4.7.2 Logging introduction

355 Here an overview on what the RaspiOS does on logging. The RaspiOS system offers a Journal service (**systemd-journald**) that collects and stores logging data. It creates and maintains structured, indexed journals based on logging information that is received from a variety of sources. This service is always present as part of the **systemd**.

Other logging services are the **Rsyslog** and **Syslog-ng**, but are optional:

- 360
- **Rsyslog**: when complex filtering and routing is required, compatibility with software requiring a traditional syslog daemon
 - **Syslog-ng** (next-gen): offers a advanced filtering and has native support for structured logging formats

Python also provides a built-in logging service for emitting log messages to track events and errors during execution. Python's **logging.handlers** module provides various handlers for directing log messages to different outputs, such as files, streams, or even remote servers. The logging outputs are configured in these handlers, such as:

- Console logging: StreamHandler
- File logging: FileHandler

- 370
- Email logging: SMTPHandler (send log to email)
 - Web logging: HTTPHandler
 - Logging Server: SocketHandler

4.7.3 Logging design choices

375 As we use Python, we should use the services offered by Python logging for all modules. It is designed to operate robustly in multi-threaded applications, Raspberry Pi hardware, and web applications (Uvicorn/FastAPI).

The module log_service.py will provide a wrapper around the Python built-in logging services, with pre-configured Logger and Handlers.

380 For all Python based logging the generic base class (**logging.Handler**) is used. Processing of the logging to the configured output (file, terminal, or syslog) may take considerable time. A queue-based handler should be used (**QueueHandler**): meaning that an emitted log-record is put on log_queue and consumed by a logging listener (**QueueListener**). In this way the logging output can not block the main application. The application only needs to enqueue the record quickly, while the
385 listener performs the slower output work in the background.

Another choice is to integrate the logs of the Web-Server (Uvicorn/FastAPI). The Web-server has its own loggers defined. Which is not convenient, as we want to have all the logging to the logger defined in log_service. These log-records are being re-routed to **QueueHandler** used in log_service.

390 See the [#Logging activity diagram](#) below for more details on design.

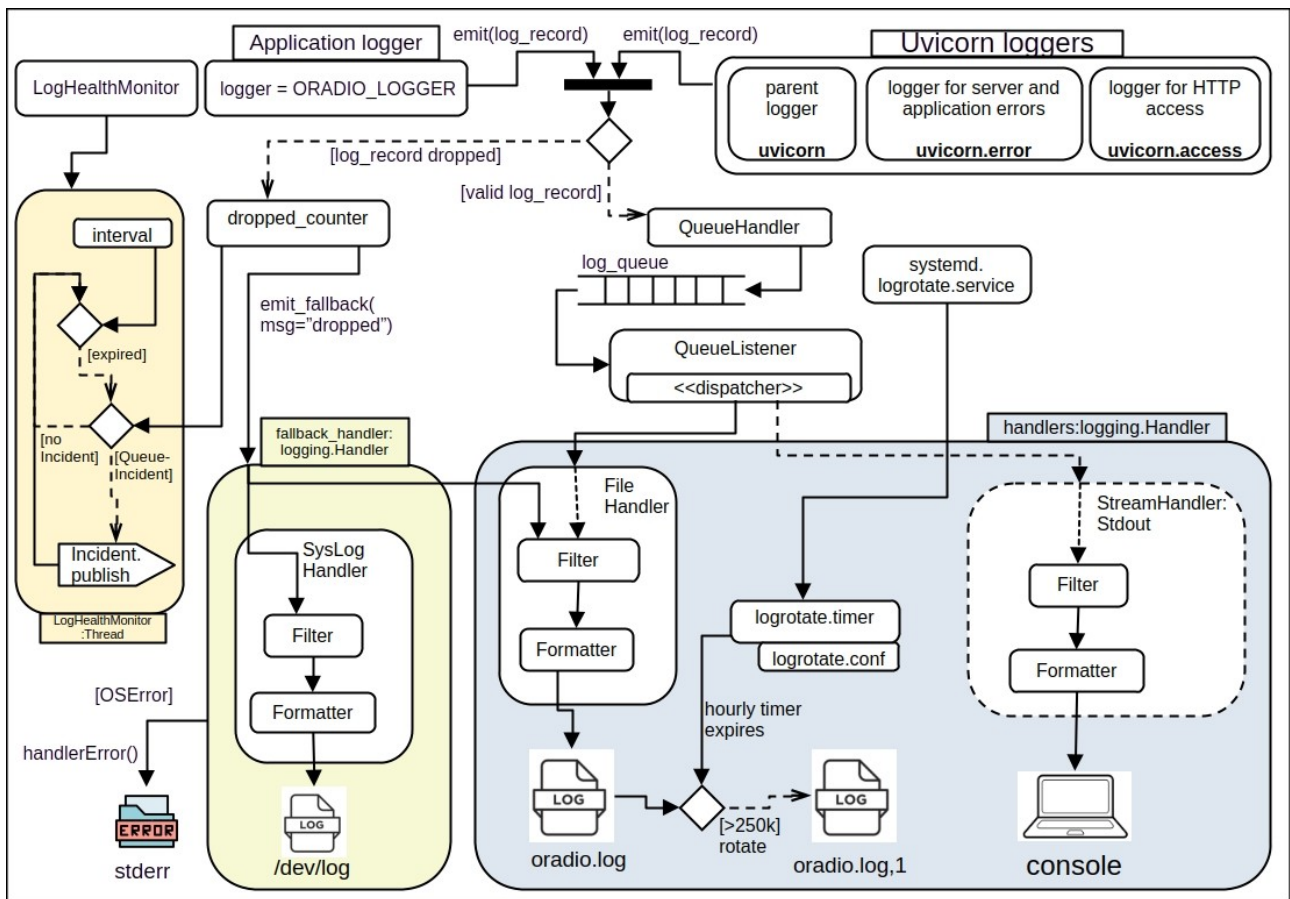


Figure 2: Logging activity diagram

4.7.4 Loggers

395 In the Oradio there are two loggers:

- The ORADIO_LOGGER as Application logger to emit log records.
- The Uvicorn loggers used by the uvicorn server to emit log records through specific named loggers, being:
 - uvicorn.error: is used during startup, shutdown, server errors, and application
 - uvicorn.access: http access logs
 - uvicorn: the parent logger as a shared aggregation point for the child loggers

400

There are 2 handlers configured for different outputs:

- a StreamHandler sending log to the standard output (stdout) as console output
- a FileHandler sending output to LOGGING_PATH/oradio.log.

405

4.7.5 Queue handling

To separate the logging output from the application logger, a queue is used. This prevents the logging output from blocking the main application. A QueueHandler and QueueListener are used:

410

- The **QueueHandler** receives log records from the python modules and from the uvicorn loggers and places them into a queue.
- The **QueueListener** reads the log record from the queue in a background thread and distributes them to the configured output handlers

4.7.6 Fallback loggers

415 In case of errors with the queue-based logging, and the Application logging was not performed, a fallback mechanism should be provided to not lose the log-record. The fallback handler should be independent of the Application logger handlers.

During emit of a log_record the **QueueHandler** will check if putting the log_record on the log_queue was successful or not. If not it will be recorded as a dropped record. The log_record will
420 be emitted to the configured fallback handlers. There are 2 fallback handlers configured:

- One handler uses the already configured **FileHandler**, for logging to the oradio.log file located at ORADIO_LOG_PATH. Now the QueueHandler is bypassed, and it tries to output the log_record directly to the oradio.log.
- Other handler is a **SysLogHandler** sending the log_record to the system's central syslog
425 server.

In case the fallback handlers can not process the log_record, the log_record will be sent to the **stderr** file, as a last resort.

430 4.7.7 LogHealthMonitor

As the log_service module may encounter issues during logging, such as a dropped log_record, it is necessary to publish these issues as incidents.

The log_service module is not designed to publish incidents. The incident handler requires the log_service also. So prevent circular imports and recursive logging's when incidents are reporting in
435 the log_service module, this LogHealthMonitor module will monitor issues related to log_service module and report them as incidents.

It will monitor when a log_record is dropped and report it as an incident.

4.7.8 Filter

440 A filter decides whether a log record should be processed. Following levels shall be configured:

Level names	level
CRITICAL	50
ERROR	40
WARNING	30
INFO	20
DEBUG	10
TRACE	5

4.7.9 Formatter

The formatter will format the log message text according a defined Oradio layout providing:

- a timestamp with milliseconds accuracy
- 445 • the module name and the line number
- The log level

- The log message
- For each log level a specific color

The following colors shall be applied:

Level names	level	Color
CRITICAL	50	Magenta
ERROR	40	Red
WARNING	30	Yellow
INFO	20	White
DEBUG	10	Grey
TRACE	5	Blue

450

4.7.10 Linux Logrotate utility

To prevent log files from consuming too much disk space, the OS-utility **Logrotate()** is used. The **Logrotate()** utility takes care for automatically rotating, compressing , and removing old logs to save disk space and keep the system organized. The utility works by periodically "rotating" log files: moving the current log to a timestamped archive file and creating a fresh log file for new entries.

455

Logrotate() uses a hierarchical configuration system with global defaults and service-specific overrides. The main configuration file is `/etc/logrotate.conf`, which sets system-wide defaults and includes service-specific configurations from `/etc/logrotate.d/`. Settings in service-specific files override global defaults.

460

The service-specific configuration for oradio is placed in `/etc/logrotate.d/oradio`, where the attributes for the `oradio.log` file are configured, such as:

- location of the log file
- size of logfile
- number of rotate files
- using compressing on the file

465

The Logrotate() service is triggered periodically with a timer specific configuration for oradio: in `/etc/systemd/system/logrotate.timer.d/oradio.conf`, which is configured for an hourly log file check.

When the `logrotate.timer` expires, `systemd` starts the associated `logrotate.service`, reads the `/etc/logrotate.conf` and the service-specific configuration files under `/etc/logrotate.d/`. Here oradio-specific config file is located. The rules defined in the config file will be checked and processed

470

475

[illegible]

480

485

- **oradio_log.trace():** log a message with TRACE(=5) severity level
- **oradio_log.debug():** log a message with DEBUG(=10) severity level
- **oradio_log.info():** log a message with INFO(=20) severity level
- **oradio_log.warning():** log a message with WARNING(=30) severity level
- **oradio_log.error():** log a message with ERROR(=40) severity level
- **oradio_log.critical():** log a message with CRITICAL(=50) severity level
- **oradio_log.set_level():** set the logging level.

490

- **logging.Logger** <<class>>: representing a logging channel, which gets a name called ORADIO_LOGGER. The SafeLogger wrapper will inherits the methods for info(), debug(), warning(), error(), setLevel() and critical()

- **logging.Handler** <<class>>: Handler instances dispatch logging events to specific destinations, such as:
 - console output, which is a subclass of the superclass **logging.StreamHandler**, which writes (formatted) log-records to a stream
 - logging to a log file, which is a subclass of the superclass **logging.FileHandler**, which writes (formatted) log-records to a disk file.
- **StrictSysLogHandler** <<class>>: is a subclass of the superclass **logging.Handler.SysLogHandler**, which writes (formatted) log-records to a syslog server or location.
- **ColorFormatter** <<class>>: is a subclass of the superclass **logging.Formatter**, which adds ANSI based colors to messages, depending on the log level
- The public instance **oradio_log** of the class **SafeLogger** will be imported from `log_service` module during initial execution of `oradio_control.py` module.
- The **faulthandler** will be enabled during initial execution.

4.7.11.3 *faulthandler*

It is important to collect information upon e.g. crashes in Python. The **faulthandler** helps in diagnosing of such crashes, by writing a traceback to the standard error device

The Python **faulthandler** is included during import of the logging module. When enabled it provides tools for diagnosing Python interpreter crashes and fatal signals. It will be registered to the Python Interpreter. When a catastrophic signal such as SIGSEGV or SIGABRT occurs, the fault handler prints a traceback to `sys.stderr` instead of leaving only an OS-level crash message. The `stderr` output is redirected to a file. It is implemented in C so it remains functional even when the interpreter is in a corrupted state.

4.7.11.4 *StrictLogHandler*

The **StrictLoghandler** (or child) inherits from the superclass **SysLogHandler**, and reuses all attributes and methods from superclass (or parent). In Python's logging framework, when `emit()` (or the handler's internal `send`) fails, the logging system usually calls `handleError()`. In the *standard* **logging.Handler** implementation, `handleError()` generally does not raise to the caller. It may print to `stderr` (depending on config) and then continue. That means your code can't reliably detect "syslog is broken" at startup. The **StrictLogHandler** subclass overrides `handleError()` to raise an `OSError`.

4.7.11.5 *Queue*

A queue is essential to separate the actual writing of a log-record to its output, from the application log-request, to prevent holding up the application. Following classes are used:

- **NonBlockingQueueHandler**: Inherits from **QueueHandler** and puts a log record on the queue, and also counts and reports when a record is dropped (queue full)

- **QueueListener**: a class that consumes log records from a queue and dispatches to one of the log handlers

4.7.11.6 log_monitor class diagram

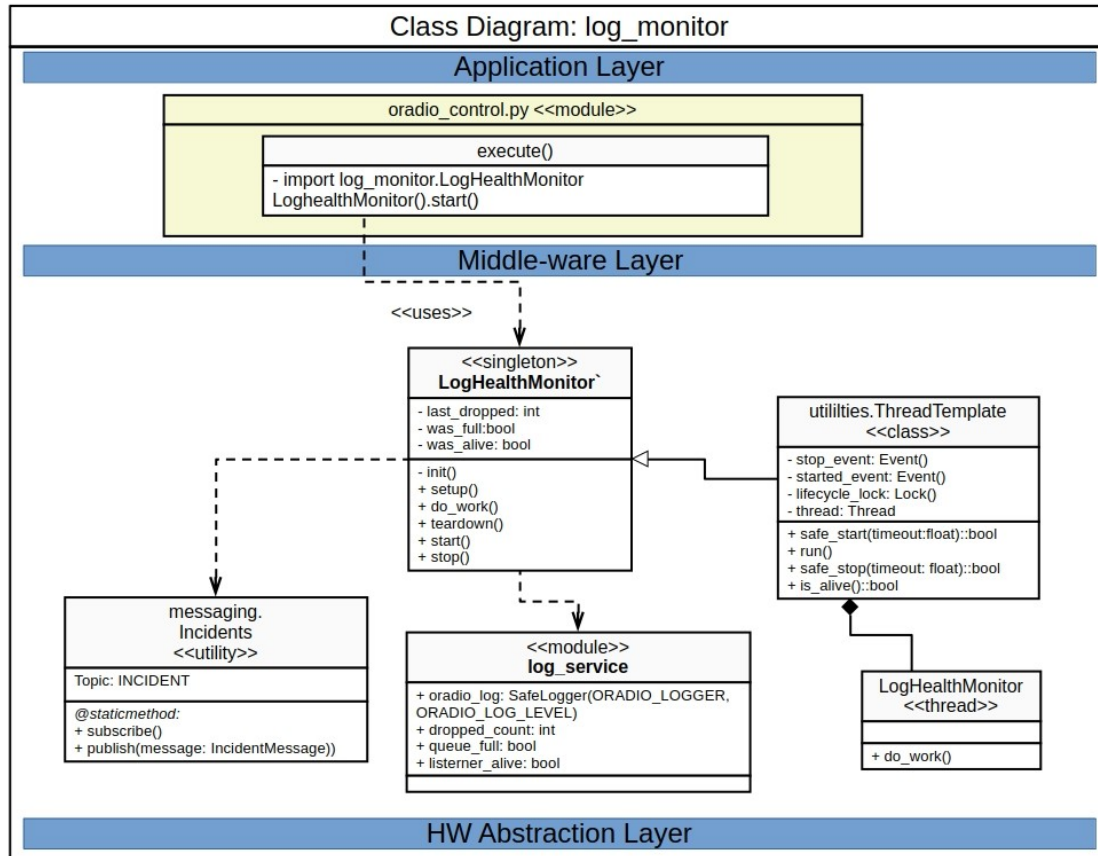


Figure 4: log_monitor class diagram

535 Reporting incidents is not possible in the **log_service** module, as incident reporting also requires the logging functionality within the **log_service**. This would lead to cyclic or recursive logging.

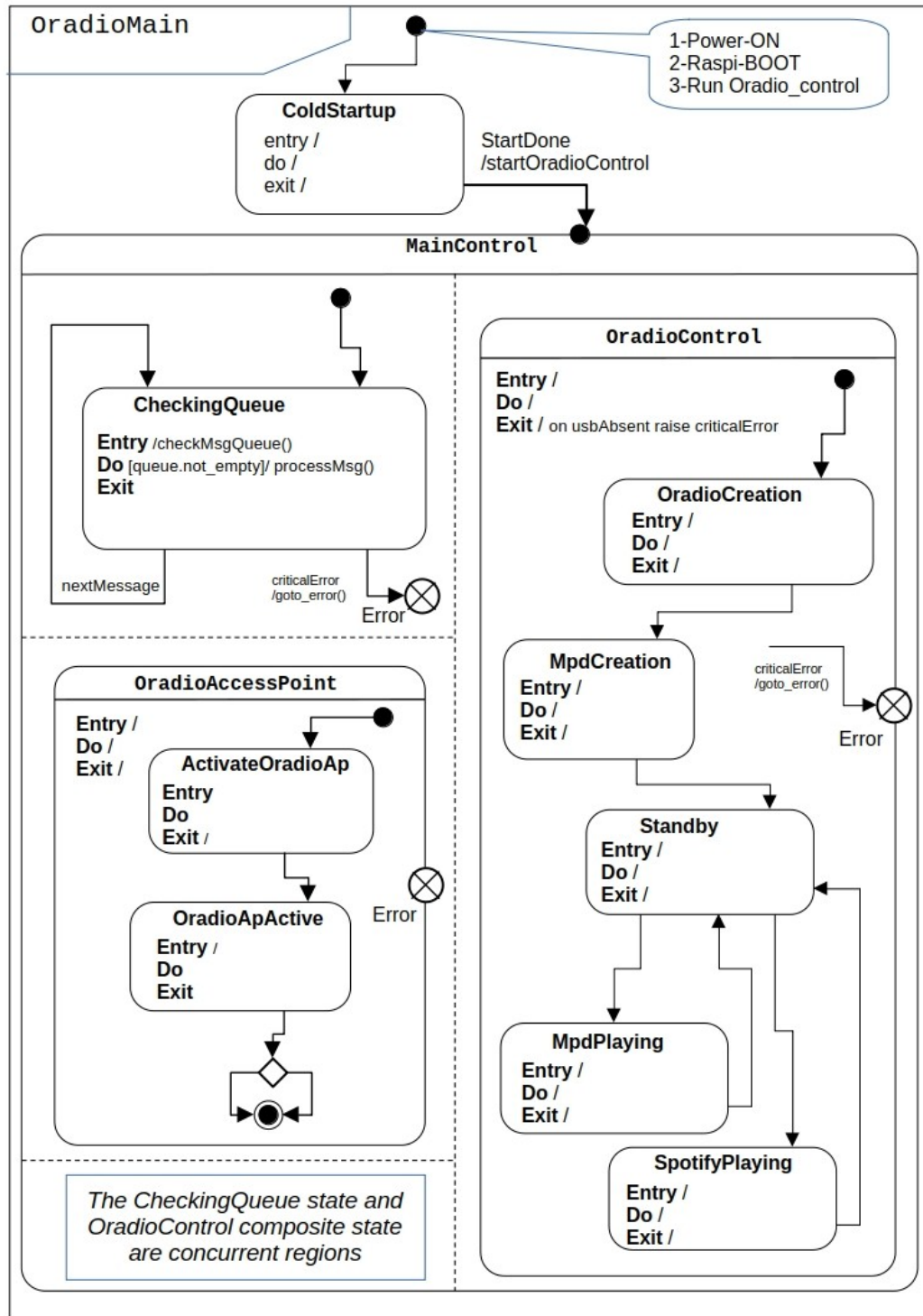
The **LogHealthMonitor** class will monitor the dropped log-records in a **LogHealthMonitor** thread. Upon a dropped log-record in `log_service` the `do_work()` method in the thread will publish an incident using the **messaging.Incident** utility

535 Reporting incidents is not possible in the **log_service** module, as incident reporting also requires the logging functionality within the **log_service**. This would lead to cyclic or recursive logging.

The **LogHealthMonitor** class will monitor the dropped log-records in a **LogHealthMonitor** thread. Upon a dropped log-record in log_service the do_work() method in the thread will publish an incident using the **messaging.Incident** utility

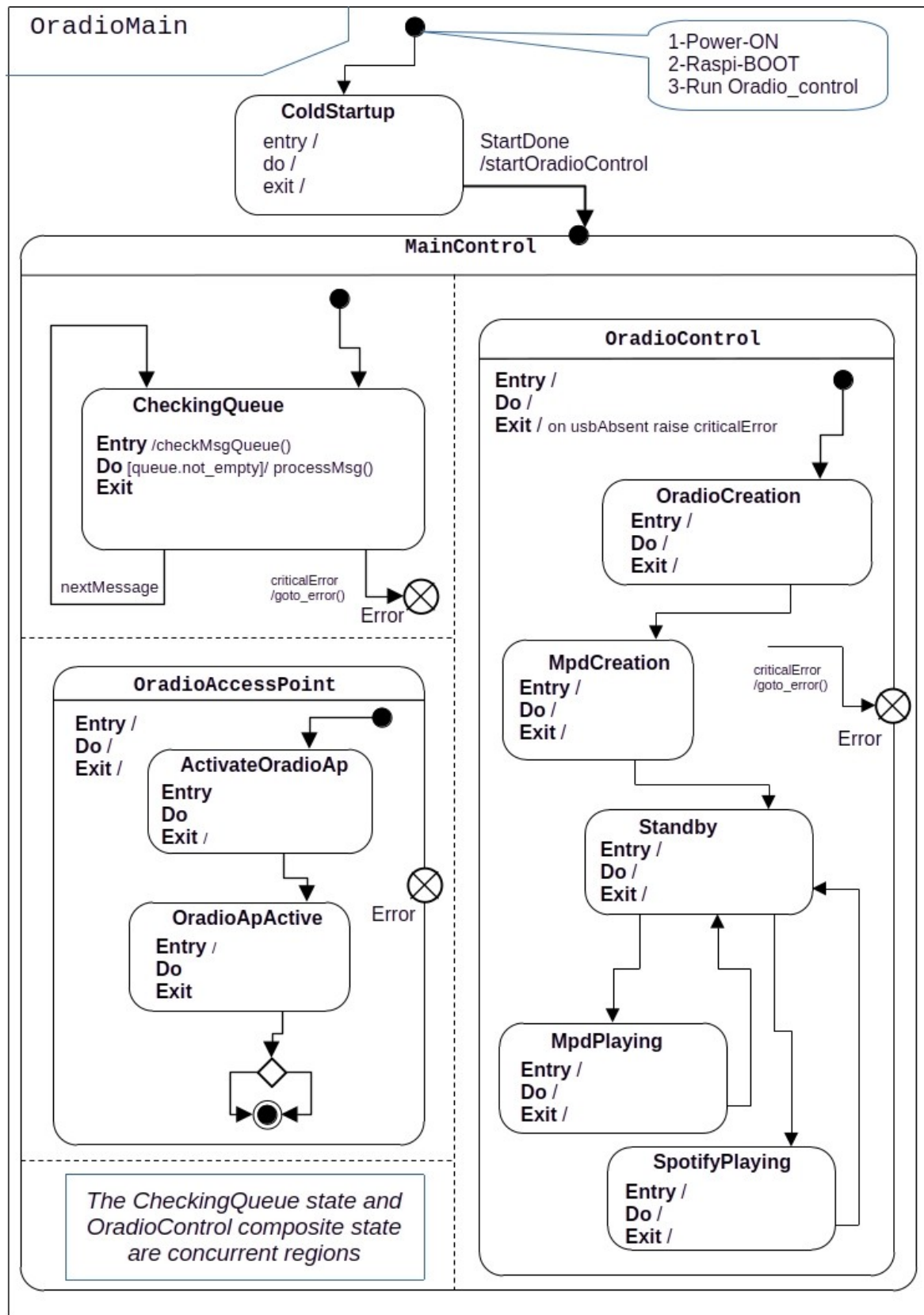
540 **5 The Logical view: State diagrams**

5.1 **Maincontrol state**



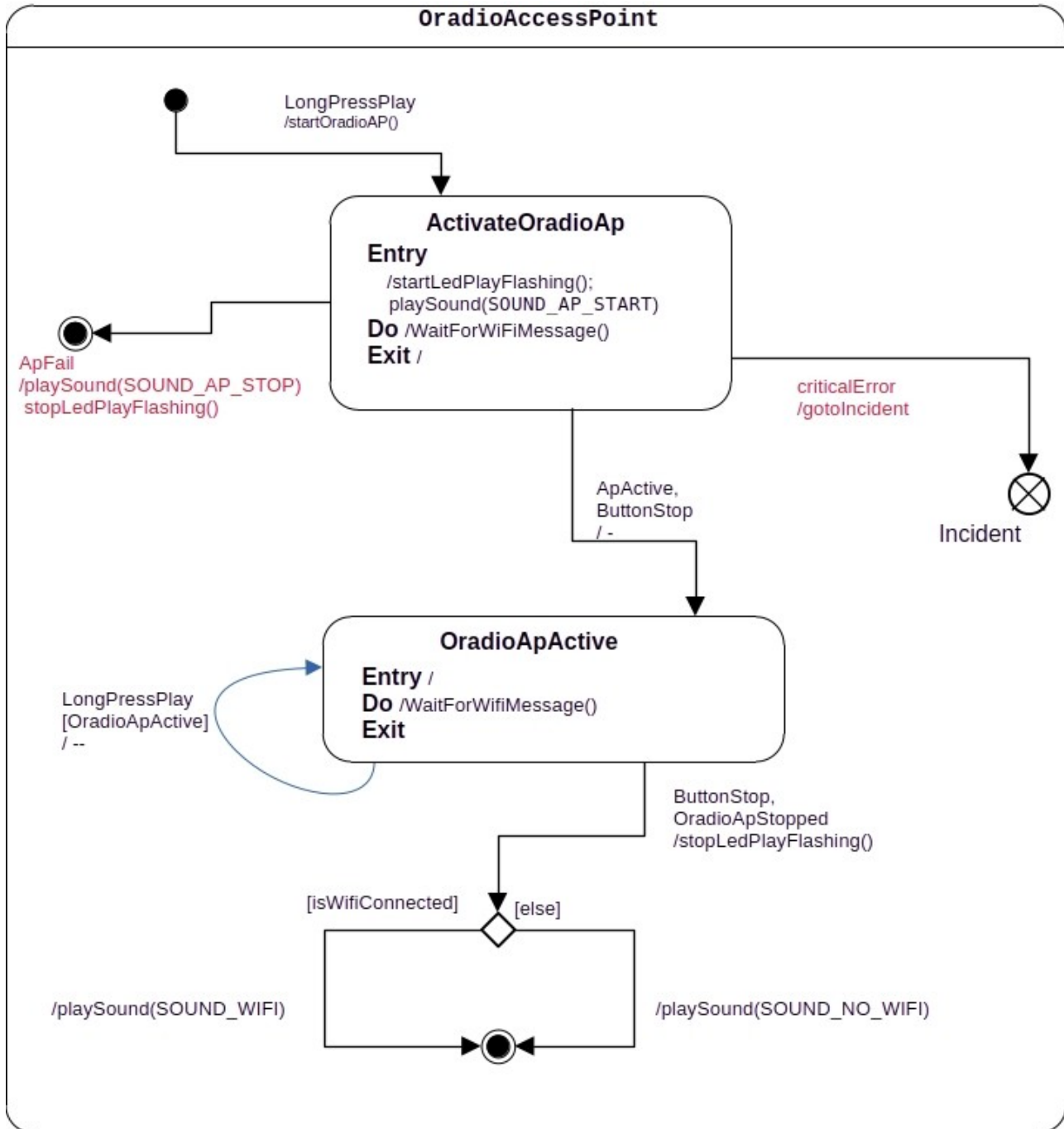
Description: xxxxxx

5.2 Oradio Control States

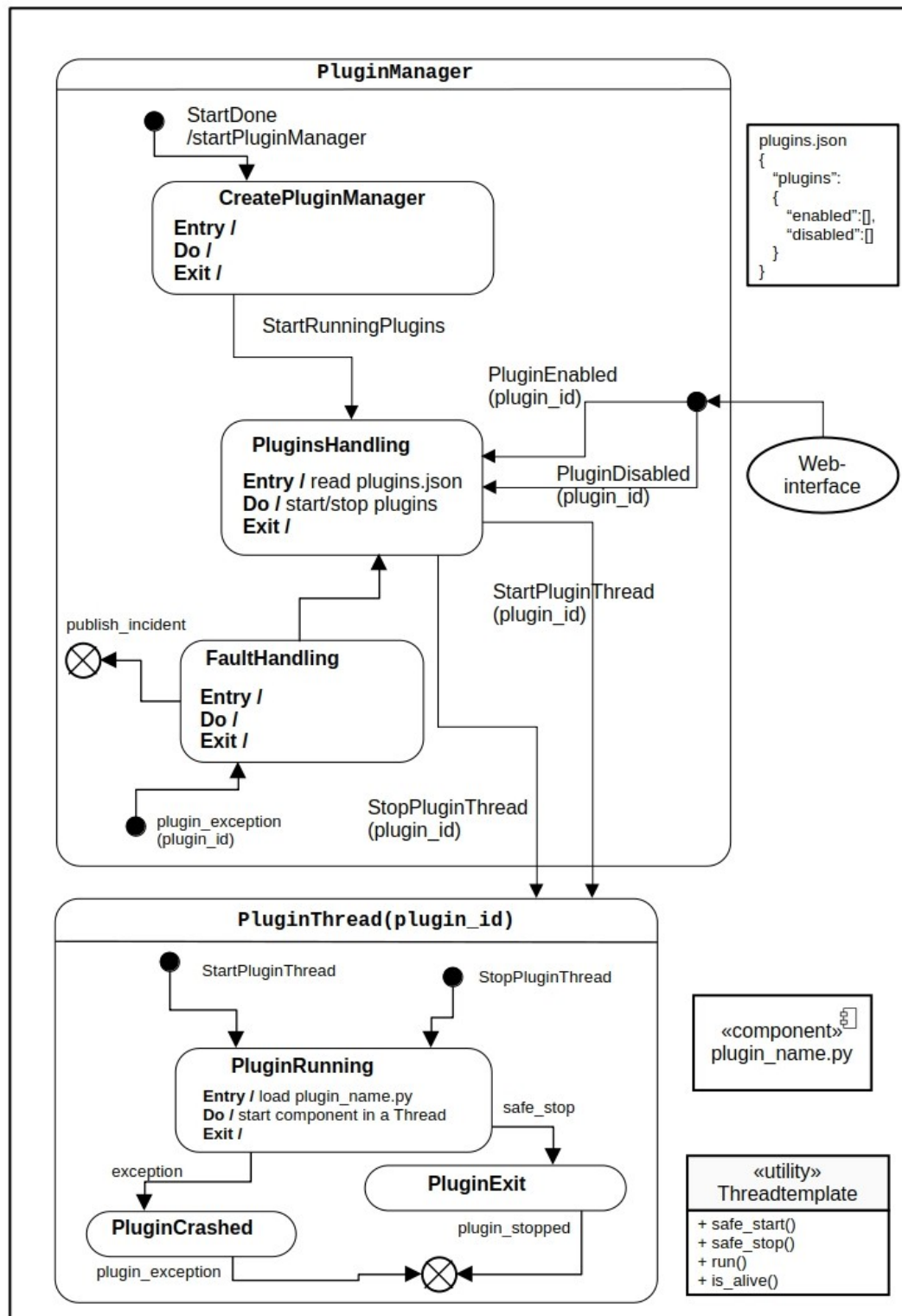


Description:xxx

5.3 Access Point state control



5.4 PluginManager State control



The plugins manager is responsible for controlling the plugins. The plugins are used to dynamical extend the functionality of the standard Oradio.

The standard Oradio is supporting:

- 555
 - Touch button handling
 - Led indications
 - System sound for announcements
 - Default music player: MPD
 - Internet access via WiFi
- 560
 - Web-interface to configure the Oradio

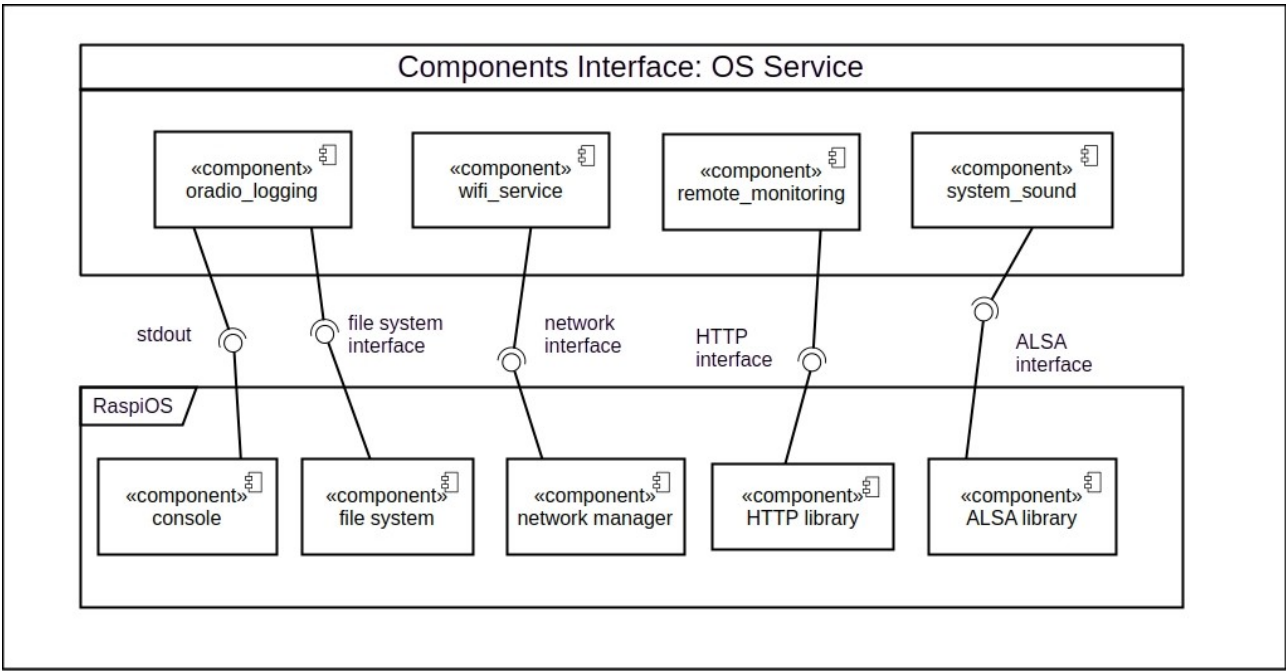
As an example the Oradio could be extended with a Spotify service. This service will be offered as plugin. The plugins manager will start/run a plugin dynamically when it is enabled, as a Thread. So it each plugin runs in its own execution context. If a plugins has a fault the PluginsManager gets an exception, so the Thread can be removed.

- 565 The plugin framework provides an abstract base class for initialization, execution and cleanup of the plugin.

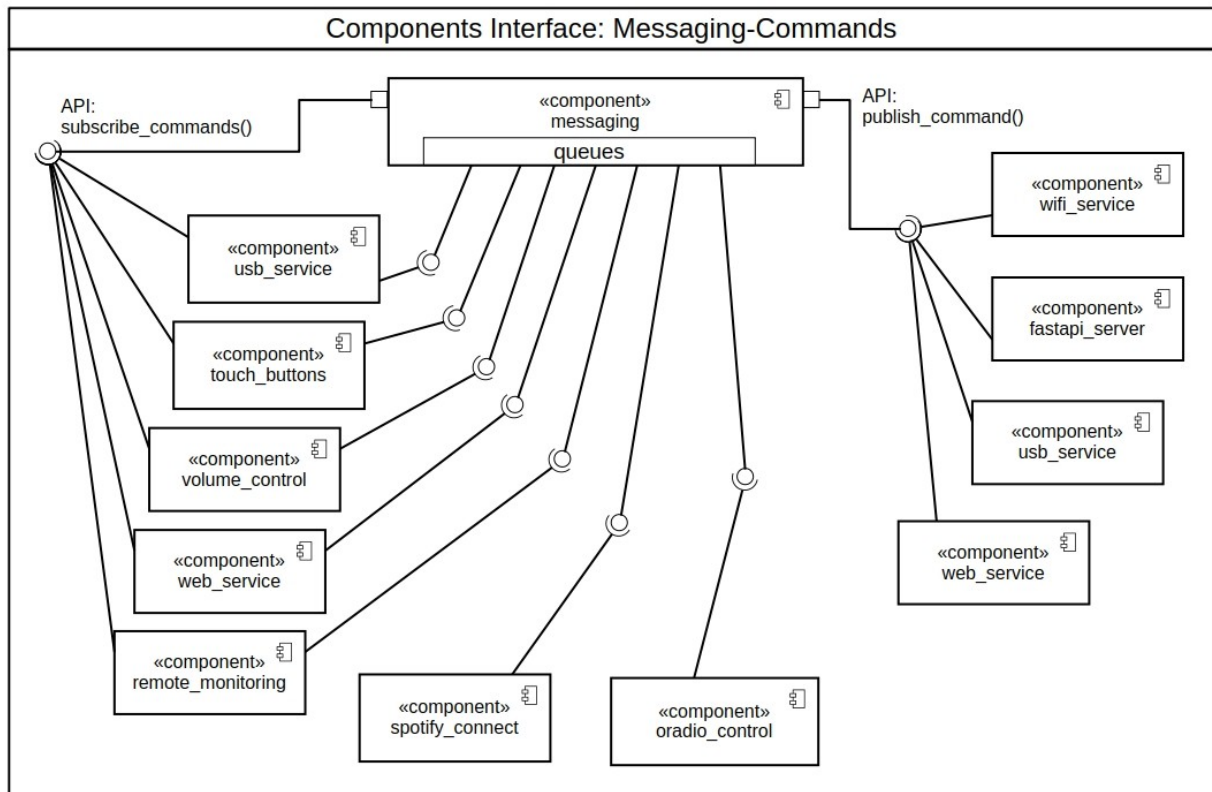
6 The Physical view: Component diagrams

570

6.1 OS Service



6.2 Messaging (Commands)

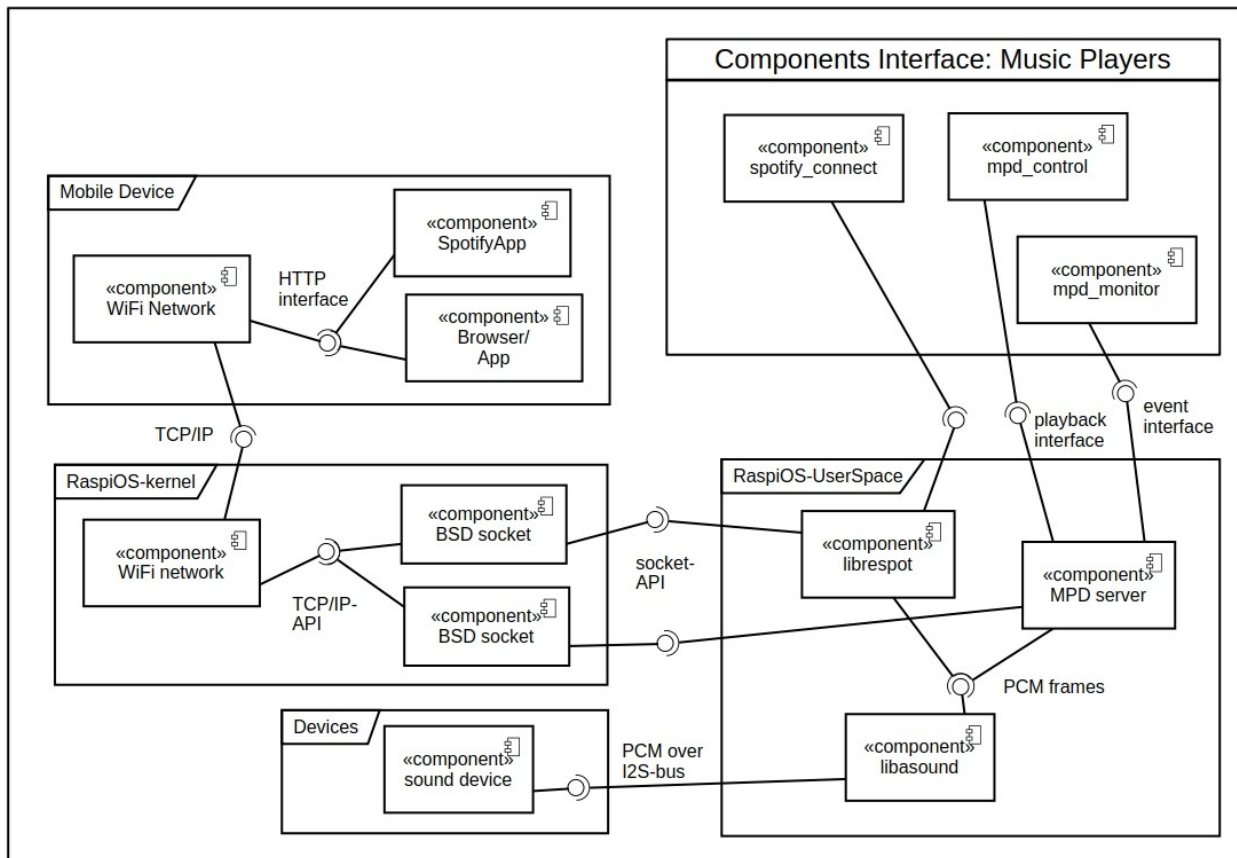


The messaging component interfaces to subscribers are queues. Each subscribers gets an assigned Command queue upon the provided `API:subscribe_commands()`. The Command queue can be monitored by the subscriber to check for new commands.

580 Commands can be published by publishers with the `API:publish_commands()`. The messaging component will distribute the published commands to all subscribers via their assigned queues.

6.3 Music Players

585



6.4 GPIO components

